Required Packages (NuGet / NPM)

Everything the base module needs, and nothing else. **Zero third-party NuGet dependencies** in the core — the libraries build on the shared framework only.

.NET — NuGet packages (4)

Package	Contains	Third-party deps	You need it when
`Pixous.AI.Abstractions`	`IChatProvider`, `ITranslationProvider`, `IConversationStore`, DTOs, factories	None	Always (pulled in transitively)
`Pixous.AI.Core`	`ChatService`, `TranslationService`, strongly-typed options, runtime selection, in-memory conversation store, settings store	None	Always
`Pixous.AI.Providers`	Built-in providers: Echo, KnowledgeBase, OpenAiCompatible, AzureTranslator, LibreTranslate + multi-key fallback, document parsers (PDF/DOCX/MD), voice providers, retry/resilience	None	Always (or a subset later)
`Pixous.AI.AspNetCore`	`AddAiSuite`, `MapAiSuiteEndpoints`, `/api/ai/*` routes, error envelope, rate-limit/auth hooks, password-gated `/settings` + `/voice`	None (uses `FrameworkReference: Microsoft.AspNetCore.App`)	Only in API hosts (your backend)

Dependency direction (one-way, no cycles):

```
Pixous.AI.AspNetCore → Pixous.AI.Providers → Pixous.AI.Core → Pixous.AI.Abstractions
```

Target framework: `net10.0` (retarget to `net8.0` with a one-line change per csproj if needed). All four pack as standard `.nupkg`:

```
dotnet pack dotnet/Pixous.AI.slnx -c Release
dotnet nuget push "dotnet/src/*/bin/Release/*.nupkg" --source &lt;your-feed&gt;
```

During active development, project references work too (what the sample does):

```
dotnet add &lt;YourApi&gt;.csproj reference &lt;path&gt;/src/Pixous.AI.AspNetCore/Pixous.AI.AspNetCore.csproj
```

Optional NuGet (consumer-side — never required by the base)

Add only what YOUR project uses; the base never hard-depends on any of these:

Package	Why you might add it
`Microsoft.Extensions.AI.Abstractions` + `.OpenAI` (10.x)	Wrap any `IChatClient` behind the chat seam
`Microsoft.SemanticKernel` + `Connectors.OpenAI` (1.80.1)	Plugins / function-calling
`Azure.AI.Translation.Text` (2.0.0)	Typed Azure Translator SDK instead of REST
`Google.Cloud.Translate.V3` (3.x)	Google Translate backend
`Azure.Identity` (1.x)	Managed Identity — no key at all
`Azure.Extensions.AspNetCore.Configuration.Secrets`	Key Vault configuration provider
`Microsoft.AspNetCore.Mvc.Testing`	Integration tests (WebApplicationFactory)

Angular — NPM packages

```
npm i @pixous/ai-suite @ngx-translate/core @ngx-translate/http-loader
```

Package	Role	Type
`@pixous/ai-suite`	`AiChatService`, `AiTranslationService`, `AiSettingsService`, `AiVoiceService`, `<pix-chat-widget>`, `<pix-language-switcher>`, auth interceptor	The library
`@ngx-translate/core`	Runtime UI string translation	Dependency of the library
`@ngx-translate/http-loader`	Async translation file loading	Dependency (used by the loader path)

Peer dependencies (your app already provides them):

Peer	Version
`@angular/common`, `@angular/core`, `@angular/forms`	^17 / ^18 / ^19
`rxjs`	^7.8

Streaming chat uses the native `fetch` API — no polyfills, no extra WebSocket/SSE packages. Typecheck the library itself: `cd angular/pixous-ai-suite && npm install && npm run typecheck`.

What you do NOT need

- No database — conversations are in-memory (or your own `IConversationStore`).
- No Redis, no message broker, no background workers.
- No secret manager package — pointers + User Secrets / env vars / Key Vault provider of your choosing.
- No HTML/Markdown renderer package on the backend — demo sanitization lives in the sample frontend; pick your own for production.